SAiDL Inductions Assignment Documentation

Samarth Bhatnagar

Contents

1 Core ML

1.1 Basic Transformer Baseline

Here, we have implemented a decoder-only Transformer model with full attention, the code follows the following architechture:

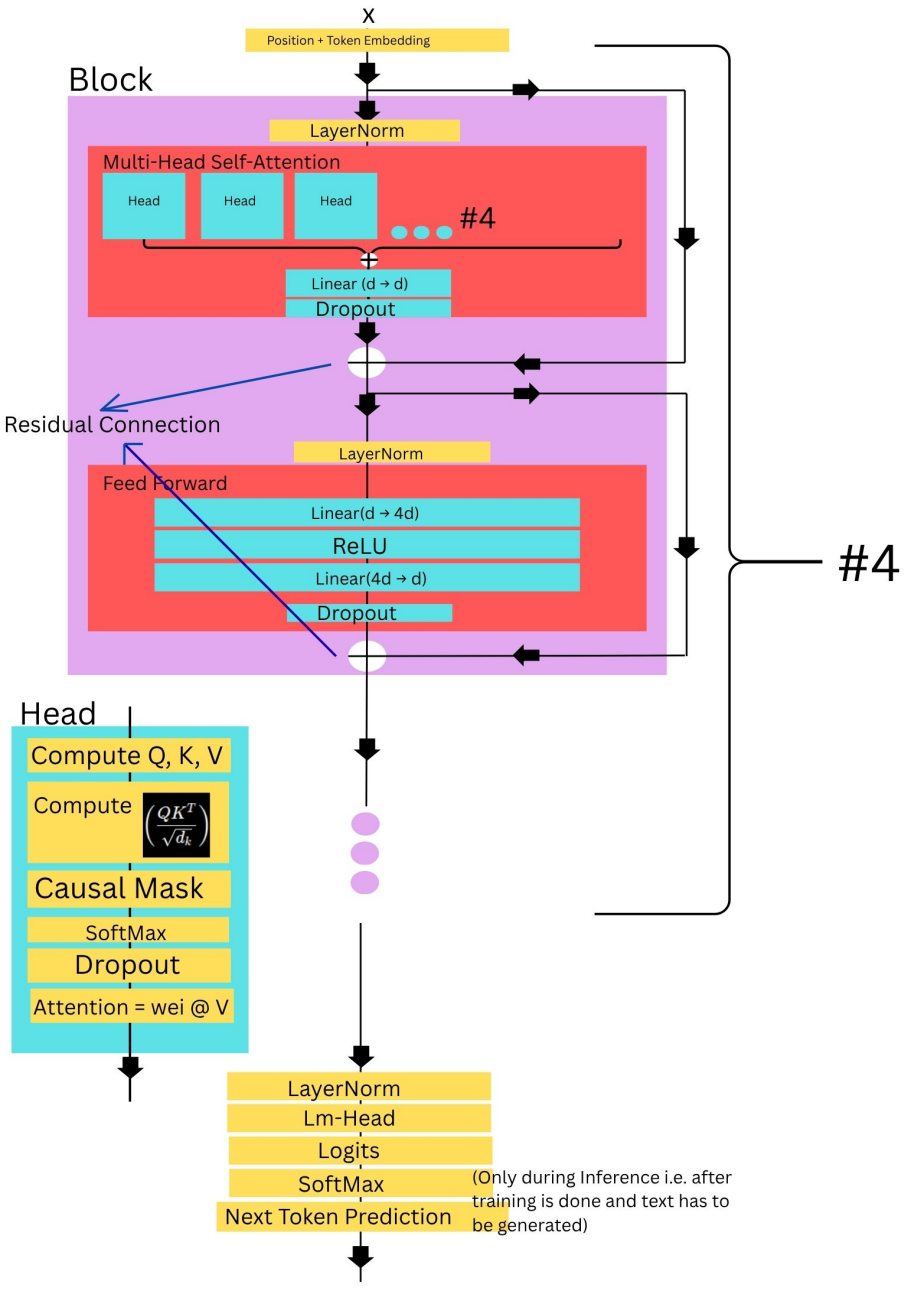

Figure 1: Architecture of the implemented Transformer language model.

The model was trained on the WikiText-2 dataset using the GPT-2 tokenizer. The end-of-sequence (EOS) token was used as the padding token (GPT2 tokenizer by default has no padding, padding is done to make the sentences of the same length). After tokenization, all examples were concatenated into a single continuous stream of tokens. During training, batches were formed by randomly sampling contiguous sequences of length L, where L denotes the context length.

The following context lengths were evaluated:

$$L \in \{256,\ 512,\ 1024\}$$

All other hyperparameters were kept fixed across experiments which were:

Table 1: Model Hyperparameters

Hyperparameter	**Value**
Embedding Dimension (n_{embd})	128
Number of Transformer Blocks	4
Number of Attention Heads	4
Dropout Rate	0.2
Batch Size	8
Learning Rate (Step size during gradient descent)	3×10^{-4}
Training Iterations	1500
Optimizer	AdamW

Two primary metrics were used for evaluation:

- **Perplexity**
  Perplexity measures the quality of a language model and is computed as:

$$PPL = e^{\text{Loss}} \tag{1}$$

  Lower perplexity values indicate better predictive performance.

- **Throughput**
  Training throughput was measured as:

$$\text{Throughput} = \frac{\text{Tokens Processed}}{\text{Elapsed Time}} \tag{2}$$

We got the following results:

Table 2: Training Results for Different Context Lengths

Context Length	**Parameters (M)**	**Val PPL**	**Throughput (tokens/s)**	**GPU Memory (GB)**
256	13.74	431.30	32615	1.76
512	13.77	404.74	29992	3.31
1024	13.84	369.11	25171	6.43

Table 3: Inference Benchmark

Context Length	Inference Tokens/s	Inference Latency
256	106.39	0.0094
512	130.04	0.0077
1024	99.19	0.0101

Here, Inference Tokens/s and Latency per token are

$$\text{Inference Tokens / s} = \frac{\text{Number of Generated Tokens}}{\text{Generation Time}}$$

$$\text{Inference Latency} = \frac{1}{\text{Inference Tokens / s}}$$

1.1.1 Observations & Conclusions

- Increasing the context length consistently improved language modeling performance.
- No. of parametres increase as context length increases.
- Training throughput decreased as the context length increased, because the computational complexity of self-attention scales quadratically with sequence length. Consequently, larger context windows require substantially more attention computations.
- GPU memory consumption increased significantly with context length.
- There is no monotonic increment or decrement in Inference Latency wrt context length, it has a minima at L = 512. Ideally it should increase as L increases, because as L increases the no. of tokens that need to be processed increases quadratically. Here, there are many more factors like hardware utilization and measurement noise which might have dropped it at L = 512. We do can see that for 1024 it again increased in comparision to that for 256.
- Text generated by the trained models are a bit random and dont really make sense which can be attributed to several factors:
  - The model contains only approximately 14 million parameters.
  - Training was limited to 1500 iterations.
  - WikiText-2 is relatively small compared to modern language-modeling datasets.

1.2 Sliding Window Attention

In the baseline Transformer, each token attends to all previous tokens within the context window, this results in a quadratic attention complexity. As the context length increases, both memory consumption and computational cost grow rapidly. To mitigate this issue, Sliding Window Attention restricts each token to attend only to a fixed number of recent tokens.

Instead of computing attention over the entire sequence, a window of size w is introduced. For a token at position i, attention is only computed over tokens in the range [i-w+1,; i] while still enforcing causal masking so that future tokens remain inaccessible.

In our implementation, the window size was fixed to (w = 64). Thus, each token can attend to at most the previous 63 tokens and itself.

In the code, only the baseline Multi-Head Self-Attention module was replaced by a Sliding Window Attention module. Apart from the attention mechanism, all remaining architectural components were kept identical to the baseline Transformer. In the code we created one matrix called "window_mask" which when multiplied by matrix "causal mask" gives the matrix "mask" which has a similar form as the above matrix but with window length 64.

Following are the matrices "causal mask", "window_mask", "mask" in our code.

$$M_{\text{causal}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$M_{\text{window}} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$M_{\text{SW}} = M_{\text{causal}} \odot M_{\text{window}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

The architechture of the head is changed only slightly by adding a sliding window mask too.

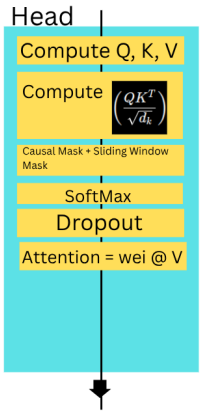

Figure 2: Head in SW Attention.

Following are the results:

Table 4: Comparison of Full Attention and Sliding Window Attention

Context	Method	Params (M)	Val PPL	Throughput (tokens/s)	Memory (GB)
256	Full Attention	13.74	431.30	32615	1.76
256	Sliding Window	13.74	423.65	34049	1.76
512	Full Attention	13.77	404.74	29992	3.31
512	Sliding Window	13.77	389.18	28981	3.30
1024	Full Attention	13.84	369.11	25171	6.43
1024	Sliding Window	13.84	345.30	23982	6.37

1.2.1 Observations & Conclusions

We can see that across all context lengths, Sliding Window Attention produces slightly lower validation perplexity which suggests that local contextual information is sufficient restricting attention does not hurt model quality.

Just like before we can see that for sliding window too the throughput decreases and the memory consumption increases as L increases.

Between full and sliding window, the memory consumption remains about the same as, in sliding window we had first made the full matrix and then performed masking.

Theoretically since the time complexity in SW is O(nw), w = window length, the throughput should increase for SW for a specific L. But we can see that for L = 512 and 1024 it rather decreased. Which can be attributed to modern GPU libraries being highly optimized for dense matrix multiplications used in Full Attention, whereas sparse attention patterns often cannot fully utilize these optimized kernels.

1.3 Linear Attention

While calculating the attention matrix Linear attention performs the computation by mapping Q and K using the map

$$\phi(x) = \text{ELU}(x) + 1, \text{ where ELU(x)} = \text{x for x} > 0 \text{ and } e^x - 1 \text{ for x} <= 0$$

and then we use the following formula

$$V_i' = \frac{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j) V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j)} \tag{3}$$

to get the representation of tokens after attention. The head in linear attention looks like:

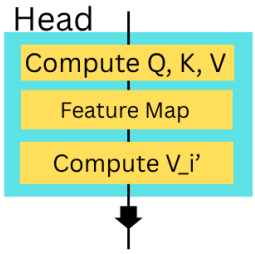

Figure 3: Head in linear attention.

The derivation of equation (3):

Standard scaled dot-product attention is given by

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Which can be written as, by opening the softmax

$$V_i' = \frac{\sum_{j=1}^{N} \exp\left(\frac{Q_i K_j^\top}{\sqrt{d_k}}\right) V_j}{\sum_{j=1}^{N} \exp\left(\frac{Q_i K_j^\top}{\sqrt{d_k}}\right)}.$$

Now we replace the exponential function with a similarity function $\text{sim}(Q_i, K_j)$

$$\text{sim}(Q_i, K_j) = \phi(Q_i)^\top \phi(K_j),$$

where ϕ is a positive feature map. In our implementation,

$$\phi(x) = \text{ELU}(x) + 1.$$

Substituting the kernel similarity into the attention expression gives

$$V_i' = \frac{\sum_{j=1}^{N} \text{sim}(Q_i, K_j)V_j}{\sum_{j=1}^{N} \text{sim}(Q_i, K_j)} = \frac{\sum_{j=1}^{N} \phi(Q_i)^\top \phi(K_j)V_j}{\sum_{j=1}^{N} \phi(Q_i)^\top \phi(K_j)}.$$

Since $\phi(Q_i)$ is independent of the summation index j, it can be factored outside the summation:

$$V_i' = \frac{\phi(Q_i)^\top \sum_{j=1}^{N} \phi(K_j)V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^{N} \phi(K_j)}.$$

For the causal language modeling setting, the summation is restricted to previous tokens only:

$$V_i' = \frac{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j)V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^{i} \phi(K_j)}.$$

In the code we have just changed the attention module which maps K and V via a feature map and computes V_i' according to the above formula.

1.3.1 Observations & Conclusions

Table 5: Comparison of Full, Sliding Window, and Linear Attention

Context	Method	Params (M)	Val PPL	Throughput	Memory (GB)	Inference TPS	Latency (s/token)
256	Full Attention	13.74	431.30	32615	1.76	106.39	0.0094
256	Sliding Window	13.74	423.65	34049	1.76	210.12	0.0048
256	Linear Attention	13.74	438.41	28385	1.76	222.46	0.0045
512	Full Attention	13.77	404.74	29992	3.31	130.04	0.0077
512	Sliding Window	13.77	389.18	28981	3.30	190.55	0.0052
512	Linear Attention	13.77	412.26	28765	3.30	200.91	0.0050
1024	Full Attention	13.84	369.11	25171	6.43	99.19	0.0101
1024	Sliding Window	13.84	345.30	23982	6.37	80.19	0.0125
1024	Linear Attention	13.84	370.61	28985	6.37	105.33	0.0095

We can see that for linear attention, increasing context length improves Val PPL. Memory usage and param count remains the same for every context length.

Here, the throughput for linear attention is almost constant largely unaffected by L. Because the computation here does not depend on the L×L attention matrix. In contrast to computation of QK^T in full attention we compute the terms given in the formula (3). The required quantities can be obtained by avoiding pairwise interactions between all token positions. As a result, the computational complexity scales approximately linearly with the sequence length. But we see that throughput goes approximately constant because other parts of the model dominate the runtime at our scale.

We can see that for a specific context length Linear attention has the lowest Latency as the computation here is minimum.

1.4 Multi-Query Attention (MQA)

In MQA, the heads share multiple queries i.e. in each head of the multi-headed MQ attention module we have (Q1, K, V), (Q2, K, V), (Q3, K, V), ...

In the code we have kept n_kv_head which is the no. of common heads with the same K and V. and we created a variable repeat_factor which tells how many replication of K and V have to be made so that the no. of K and V match the no. of Q's.

Since many K and V have been repeated the param count decreases.

1.4.1 Observations & Conclusions

Table 6: Comparison of Attention Mechanisms

Context	Method	Params (M)	Val PPL	Throughput	Memory (GB)	Inference TPS	Latency (s/token)
256	Full Attention	13.74	431.30	32615	1.76	106.39	0.0094
256	Sliding Window	13.74	423.65	34049	1.76	210.12	0.0048
256	Linear Attention	13.74	438.41	28385	1.76	222.46	0.0045
256	MQA	13.64	436.40	35659	1.76	220.26	0.0045
512	Full Attention	13.77	404.74	29992	3.31	130.04	0.0077
512	Sliding Window	13.77	389.18	28981	3.30	190.55	0.0052
512	Linear Attention	13.77	412.26	28765	3.30	200.91	0.0050
512	MQA	13.68	395.48	29920	3.30	177.97	0.0056
1024	Full Attention	13.84	369.11	25171	6.43	99.19	0.0101
1024	Sliding Window	13.84	345.30	23982	6.37	80.19	0.0125
1024	Linear Attention	13.84	370.61	28985	6.37	105.33	0.0095
1024	MQA	13.74	349.32	26006	6.38	99.12	0.0101

For every attention mechanism, validation perplexity decreases as context length increases.

Linear Attention gives the best inference efficiency for a specific context length.

Overall Sliding window attention has the best Val PPL for every context length amongst all the attention mechanisms.

MQA achieves the best perplexity at longer contexts, which suggests that sharing Keys and Values does not hurt language modeling performance and acts as some sort of a regularizer that improves generalization.

Better memory efficiency than Full Attention, which is more visible for long context lengths.

1.5 RoPE

In the code, we implemented RoPE by modifying the Q, K vectors before the attention computation. For each token position p, we generated sinusoidal frequencies

$$\theta_i = 10000^{\frac{-2i}{d}}$$

where d is the head dimension and i denotes the embedding pair index. These frequencies were used to rotate pairs of dimensions in the Q, K vectors through a position-dependent transformation:

$$\mathbf{x}' = \mathbf{x}\cos(p\theta) + \text{rotate\_half}(\mathbf{x})\sin(p\theta)$$

The rotated Q, K were then used to compute the attention scores QK^T. By encoding positional information through rotations, the dot product between two tokens naturally depends on their relative distance, allowing the model to capture positional relationships without explicit position embeddings. Since these rotations are computed analytically rather than learned, RoPE introduces no additional trainable parameters while enabling strong generalization to longer sequence lengths.

1.5.1 Observations & Conclusions

Table 7: RoPE Results

Metric	Value
Parameters (M)	13.71
Final Train PPL	236.58
Final Validation PPL	317.59
Training Throughput (tokens/s)	27252
GPU Memory (GB)	3.55
Inference Tokens/s	66.12
Inference Latency (s/token)	0.0151

Table 8: RoPE Context Length Extrapolation

Training Context	Test Context	Validation PPL
512	512	330.78
512	1024	330.80
512	2048	342.99

The model has 13.71M parameters, which is essentially unchanged from the baseline transformer because RoPE introduces positional information through rotations rather than additional learnable parameters. This makes it a parameter-efficient positional encoding technique.

The final training perplexity is 236.58, while validation perplexity at the training context length (512) is 330.78. The gap between train and validation perplexity suggests some degree of overfitting, but the model still generalizes reasonably well to unseen validation data.

the val PPL remains more or less constant for different test context lengths thus the model can generalize to sequence lengths much longer than those seen during training without requiring retraining or architectural changes.

The training throughput (27,252 tokens/s) and memory usage (3.55 GB) are very close to the baseline transformer. This indicates that the additional rotary computations introduce negligible overhead, making RoPE a computationally efficient positional encoding scheme.

Although RoPE achieves strong context extrapolation, the inference throughput (66.12 tokens/s) is lower than that observed for ALiBi in your experiments. This suggests that while RoPE provides excellent positional modeling, its runtime efficiency may be slightly inferior to simpler bias-based approaches.

1.6 Relative Positional Encoding

In the code, we implemented Relative Positional Encoding by introducing a learnable embedding table that stores representations for all possible relative distances between tokens. During attention computation, we first calculated the standard attention scores QK^T using the Query and Key vectors. We then computed the relative distance $(j - i)$ between every pair of token positions and retrieved the corresponding relative embedding r_{i-j}. Additional position-based attention scores were computed as

$$\text{RelScore}_{ij} = q_i^T r_{i-j}$$

where q_i is the query vector of token i. These relative scores were added to the standard attention scores, giving

$$\text{score}_{ij} = q_i^T k_j + q_i^T r_{i-j}$$

before applying the softmax operation. This allows the model to incorporate both content information and relative positional relationships when determining attention weights. Unlike RoPE and ALiBi, Relative Positional Encoding introduces additional learnable parameters through the embedding table, enabling the model to explicitly learn distance-dependent attention patterns.

1.6.1 Observations & Conclusions

Table 9: Relative Positional Encoding Results

Metric	Value
Parameters (M)	15.80
Final Train PPL	219.36
Final Validation PPL	316.74
Training Throughput (tokens/s)	19728
GPU Memory (GB)	3.58
Inference Tokens/s	46.40
Inference Latency (s/token)	0.0216

Table 10: Relative Positional Encoding Context Length Evaluation

Training Context	Test Context	Validation PPL
512	512	307.33
512	1024	322.96
512	2048	319.96

The train PPL decreases from 236.58 (RoPE) to 219.36 (Relative PE). This indicates that the learnable relative position embeddings give the model greater flexibility to capture position-dependent patterns in the training data. Although Relative PE is computationally more expensive, it achieves a slightly lower val PPL (316.74 vs 317.59 for RoPE). This suggests that explicitly learning relative distance representations can provide a modest improvement in language modeling quality.

The param count is more here because it introduces a learnable embedding table that stores representations for all possible relative token distances. In our implementation, these additional positional embeddings contribute approximately 2.1M extra parameters compared to RoPE, which uses fixed sinusoidal rotations and therefore adds no trainable parameters.

The throughput here is much less in comparision to that of RoPE, cause computing relative position biases incurs additional attention computation, reducing both training and inference throughput.

The val PPL doesnt really change much for different test context lengths which indicates strong context-length generalization.

1.7 ALiBi

In the code, we implemented Attention with Linear Biases (ALiBi) by adding a position-dependent bias directly to the attention scores rather than using positional embeddings. For each attention head, a fixed slope m_h was assigned using the ALiBi slope generation method, allowing different heads to model dependencies at different distance scales. During attention computation, we first calculated the standard attention scores $QK^T/\sqrt{d_k}$ and then computed the relative distance $(i - j)$ between every pair of token positions. A linear bias was generated as

$$\text{Bias}_{ij} = -m_h(i - j)$$

where m_h is the head-specific slope. The final attention score therefore becomes

$$\text{score}_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}} - m_h(i - j)$$

This bias was added before applying the softmax operation, encouraging the model to naturally prefer nearby tokens while still allowing attention to more distant tokens when necessary. Since ALiBi uses fixed linear biases instead of learnable positional embeddings, it introduces no additional trainable parameters while still enabling effective long-context generalization.

1.7.1 Observations & Conclusions

Table 11: ALiBi Results

Metric	Value
Parameters (M)	13.71
Final Train PPL	222.28
Final Validation PPL	303.31
Training Throughput (tokens/s)	27677
GPU Memory (GB)	3.55
Inference Tokens/s	103.17
Inference Latency (s/token)	0.0097

Table 12: ALiBi Context Length Evaluation

Training Context	Test Context	Validation PPL
512	512	317.57
512	1024	313.40
512	2048	318.05

We can see that the model has 13.71M parameters, identical to the RoPE model and very close to the baseline transformer cause it introduces positional information through fixed linear biases rather than learnable embeddings, resulting in zero additional trainable parameters.

Best validation perplexity among positional encoding methods.

ALiBi is the fastest since it only adds a simple bias matrix to attention scores which leads to negligible computational overhead.

It has the best throughput in comparision to other methods since there is not much extra computation being done here. It also has the best Inference Latency.

It also generalizes effectively to contexts significantly longer than those seen during training.

1.8 Convolution Design 1: Conv1D layer before each attention block

From the above results of different attention mechanisms and different positional encoding mechanisms we can see that the best val PPL is among sliding window attention and ALiBi. Hence our conformers include these 2 methods.

Convolutions help the model learn local patterns by looking at a small neighborhood of tokens at a time instead of the entire sequence. A filter slides across the input and extracts useful features such as short-range dependencies, word combinations, and phrase-level structures.

In our code our pipeline of convolution is depthwise→pointwise→GeLU activation→dropout.

The depthwise convolution applies a separate convolutional filter to each embedding channel independently, allowing the model to capture local patterns from neighboring tokens. For a kernel size k, the output at position t is computed as

$$y_t = \sum_{i=0}^{k-1} w_i x_{t+i-\lfloor k/2 \rfloor}.$$

After local features are extracted, a pointwise (1×1) convolution mixes information across embedding dimensions. For each token position, it performs a linear transformation

$$y = Wx + b,$$

where W combines features from different channels. Together, depthwise and pointwise convolutions efficiently capture both local contextual information and cross-channel interactions while using significantly fewer parameters than a standard convolution.

GELU introduces non-linearity, allowing the network to learn more complex patterns. It's formula is

$$\text{GeLU}(X) = x\Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution.

1.8.1 Observations & Conclusions

Table 13: Conformer + Sliding Window Attention + ALiBi Results

Metric	Value
Parameters (M)	14.04
Final Train PPL	11.95
Final Validation PPL	12.65
Training Throughput (tokens/s)	20643
GPU Memory (GB)	3.55
Inference Tokens/s	81.56
Inference Latency (s/token)	0.0123

Table 14: Conformer Context Length Evaluation

Training Context	Test Context	Validation PPL
512	512	12.52
512	1024	15.17
512	2048	16.44

We ran the model for only 600 iterations and had encountered a val PPL of 12.65 which is very much lower than models implementing individually SW and ALiBi. We can also see that by just a small increase in the param count (14.04 - 13.71 = 0.33M) we still got such a strong performance.

*** One big challenge that I incurred here was that despite val PPL being so low (for 1500 it hitted as low as 1-2), the generated text was still not making sense. Which lead me think and try to debug that maybe something is wrong with the code. Ultimately, which made me realize that val PPL is a local predicition metric and cannot tell if the paragraph generated by the model is coherent. Since conformers significantly improve local dependencies than val PPL shall increase alot. Hence what I learnt was that val PPL is not the sole thing that can tell if the generated text will be coherent and sensical.

1.9 Convolution Design 2: Gated convolutional feedforward networks

Here, in the code we have in addition to the previous conformer added convolutions in the feedforward networks (i.e. there are are convolutions before attention heads and in the ffw's). In the feedforward gates only depthwise convolution is there. Because, during feedforward the tokens dont talk to each other(no attention) and rather go through expansion, ReLU and compression.

We have ran the model for 800 iterations.

1.9.1 Observations & Conclusions

Table 15: Gated Convolutional Feed Forward Network + Sliding Window Attention + ALiBi Results

Metric	Value
Parameters (M)	14.05
Final Train PPL	5.07
Final Validation PPL	5.41
Training Throughput (tokens/s)	16180
GPU Memory (GB)	3.30
Inference Tokens/s	73.70
Inference Latency (s/token)	0.0136

Table 16: Context Length Extrapolation for GCFFN Model

Training Context	Test Context	Validation PPL
512	512	5.42
512	1024	5.48
512	2048	5.42

Till step 600:

Table 17: Gated Convolutional Feed Forward Network + Sliding Window Attention + ALiBi Results

Metric	Value
Train PPL	10.31
Validation PPL	10.89
Training Throughput (tokens/s)	20087
GPU Memory (GB)	3.30

Here, just like in the previous conformer, the val PPL has dropped significantly and from the output of the code at step 600 we got val PPL of 10.89 which in comparision to val PPL at step 600 of the previous conformer 12.65 is a bit less. Which can be attributed to the additional convolutions in the ffw's.

Comparing the throughputs of both the conformers at step 600 we'll get the throughput of the previous design being better since there are comparatively less computations being done because of addition of convolutions in the second design.

2 Sparsity & Optimization

2.1 LoRA v/s AdaLoRA v/s SoRA

2.1.1 Observations & Conclusions

Table 18: Comparison of LoRA, AdaLoRA and SoRA on CoLA using DeBERTaV3-base

Method	MCC	Trainable Parameters	Average Effective Rank	Training Time (min)
LoRA	0.6800	296,450	7.88	9.31
AdaLoRA	0.6586	444,194	8.00	10.08
SoRA	0.2128	296,642	8.00	11.68

We can see that LoRA achieved the best overall performance. It obtained the highest MCC score (0.6800) and used only 296,450 trainable parameters, making it both parameter-efficient and effective.

AdaLoRA did not perform better in comparision to LoRA in any metric.

SoRA achieved the lowest MCC score (0.2128) despite having a similar number of trainable parameters as LoRA. In addition, it required the longest training time (11.68 minutes), indicating that the added gate-based sparsification mechanism introduced extra computational overhead without improving performance on the CoLA task.

*** A key challenge during the implementation of AdaLoRA was evaluating the effective rank after training. Initial attempts to inspect rank-related parameters such as 'lora_E' and 'ranknum' suggested that all layers retained the initial rank of 12, making it appear that adaptive rank allocation had not occurred despite configuring a target rank of 8. Further digging deeper revealed that the dynamically allocated ranks were actually stored in the 'rank_pattern' attribute of the PEFT configuration. By analyzing these rank masks and computing the number of active rank components for each adapted layer, the true effective ranks could be obtained. This debugging process was essential for correctly verifying AdaLoRA's adaptive pruning and rank allocation behavior.

*** The main challenge in implementing SoRA was that, unlike LoRA and AdaLoRA, there is no native SoRA implementation available in the PEFT library. Therefore, an approximate implementation inspired by the original paper had to be developed on top of the existing LoRA framework. This required modifying the low-rank adaptation mechanism to include learnable gate parameters and incorporating sparsity regularization during training. Another difficulty was ensuring that the gate parameters were correctly integrated into the low-rank computation rather than merely being registered as trainable parameters. Considerable effort was also spent debugging the interaction between the sparse gates, regularization term, and effective-rank computation to ensure that the implementation captured the core ideas of the original SoRA method.